

06 - Agent Context Schema Spec

Visor DOM-to-Agent Context Compiler
DOC-06 | 0.1 requirements pack | Generated 2026-06-14

This document defines the stable AgentContext output schema, field semantics, versioning rules, and example output contract.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-06
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. Schema goals

AgentContext is the stable output contract for Visor. It should be easy for an LLM, automation agent, RAG pipeline, or developer tool to consume. The schema must be versioned, validated, and backward compatible across minor releases.

2. Top-level schema

```
type AgentContext = {
  schemaVersion: 'agent_context.v1';
  source: SourceInfo;
  pageClassification: PageClassification;
  summary: ContextSummary;
  hierarchy: HeadingNode[];
  mainContent: ContentBlock[];
  actionableElements: ActionElement[];
  dataElements: DataElement[];
  links: LinkElement[];
  forms: FormElement[];
  tables: TableElement[];
  media: MediaElement[];
  tokenProfile: TokenProfile;
  privacyReport: PrivacyReport;
  compilerNotes: CompilerNote[];
};
```

3. SourceInfo

```
type SourceInfo = {
  url: string;
  canonicalUrl?: string;
  title: string;
  capturedAt: string;
  language?: string;
  contentHash?: string;
};
```

4. ActionElement

```
type ActionElement = {
  id: string;
  type: 'button' | 'link' | 'input' | 'select' | 'textarea' | 'form';
  label: string;
  selectorHint: string;
  textContext: string;
  actionPurpose: string;
  confidence: number;
  disabled?: boolean;
  required?: boolean;
  privacySensitive?: boolean;
};
```

5. ContentBlock

```
type ContentBlock = {
```

```

    id: string;
    kind: 'heading' | 'paragraph' | 'list' | 'quote' | 'code' | 'status' | 'error' | 'other';
    text: string;
    headingPath: string[];
    selectorHint?: string;
    importanceScore: number;
    tokenEstimate: number;
    sourceOrder: number;
  };

```

6. Form and table schemas

```

type FormElement = {
  id: string;
  selectorHint: string;
  label?: string;
  purpose?: string;
  fields: FormField[];
  submitControls: ActionElement[];
};

type FormField = {
  id: string;
  name?: string;
  type: string;
  label?: string;
  placeholder?: string;
  required?: boolean;
  disabled?: boolean;
  redactedValue?: string;
};

type TableElement = {
  id: string;
  caption?: string;
  headingPath: string[];
  headers: string[];
  rows: string[][];
  truncated?: boolean;
};

```

7. PrivacyReport

```

type PrivacyReport = {
  riskLevel: 'low' | 'medium' | 'high';
  redactionLevel: 'low' | 'medium' | 'strict';
  redactedItems: Array<{
    type: 'email' | 'phone' | 'password' | 'api_key' | 'jwt' | 'credit_card_like' | 'token'
    | 'other';
    count: number;
    locations: string[];
  }>;
  warnings: string[];
  externalSharingAllowed: boolean;
};

```

8. Example minimal output

```

{
  "schemaVersion": "agent_context.v1",
  "source": { "url": "https://example.com", "title": "Example", "capturedAt":
    "2026-06-14T08:00:00.000Z" },
  "pageClassification": { "type": "article", "confidence": 0.82 },
  "summary": { "short": "Example page about the product.", "method": "extractive" },
  "hierarchy": [],
  "mainContent": [],
  "actionableElements": [],
  "dataElements": [],
  "links": [],
  "forms": [],
  "tables": [],
  "media": [],
  "tokenProfile": { "compiledEstimatedTokens": 350, "budget": 4000, "budgetStatus":
    "under_budget" },

```

```
    "privacyReport": { "riskLevel": "low", "redactionLevel": "medium", "redactedItems": [],  
      "warnings": [], "externalSharingAllowed": true },  
    "compilerNotes": []  
  }  
}
```

9. Versioning rules

- Use semantic schema names such as agent_context.v1, agent_context.v1.1, agent_context.v2.
- Minor versions may add optional fields but must not remove required fields.
- Major versions may restructure fields and require migration utilities.
- Every schema change must include validator updates and fixture snapshots.